

BOOSTX

MVP Build Specification

Guest-checkout SMM panel — TikTok, Instagram, Facebook, X, Telegram

Prepared for: Enock — Computer Science, University of Ghana

Currency: Ghana Cedi (GHS) only

Fulfillment provider API: boostcenter2.com/api/v2

Document scope: architecture, database, pricing engine, provider integration, security, and phased build plan.

Table of Contents

1. Executive Summary	3
2. The Five Non-Negotiable Rules	3
3. Customer Journey	4
4. Accounts & Authentication	4
5. System Architecture	6
6. Technology Stack	7
7. Database Schema	7
8. Pricing Engine — GHS Only	8
9. Provider API Integration	9
10. Payment & AI Verification Flow	10
11. Order Lifecycle & Money Safety	12
12. Reliability Principle — What BoostX Defends Against	13
13. Admin Dashboard	13
14. API Endpoints	14
15. Security Requirements	15
16. MVP Build Phases	15
17. Critical Test Cases	16
18. Environment Variables Checklist	17
Appendix A. Code Delivered With This Specification	17

1. Executive Summary

BoostX is a guest-checkout social media marketing (SMM) panel. A customer chooses a platform and service, enters a target link or username, pays by mobile money, uploads a payment screenshot, and receives an AI-verified spendable balance in Ghana Cedis (GHS). The order is then submitted to a fulfillment provider through a REST API, and the customer tracks delivery in real time — no account or password required.

This document restructures the original BoostX specification into an implementation-ready build guide. It covers the customer journey, system architecture, database schema, the pricing engine, the fulfillment-provider integration (boostcenter2.com/api/v2), payment verification, security requirements, and a phased MVP build plan.

Scope of this revision

Pricing	Currency is GHS only. Customer price = provider cost (converted to GHS) + a flat GHS 5.00 markup, applied once per order.
Provider API	Confirmed live against boostcenter2.com/api/v2 — a standard action-based SMM provider API (services, add, status, refill, cancel, balance).
Accounts	Registration and login are new and optional. Customers and admins sign in on one shared page with no visible admin path; registering lets a customer track delivery status and full order history.
Reliability	BoostX does not promise zero issues — no honest system can. Section 12 indexes the specific failure modes it does defend against (double orders, provider failure, race conditions, fake screenshots) and names the gap that still needs pre-launch testing against the live provider.

2. The Five Non-Negotiable Rules

Rule 1 — Platform scope

Only five platforms: TikTok, Instagram, Facebook, X, Telegram. No others are shown to customers, regardless of what the provider catalog offers — confirmed against a live reference storefront (baloonboost.com/order), which lists nine (also Spotify, Audiomack, Threads, YouTube). BoostX deliberately shows five, not whatever the provider happens to support.

Rule 2 — Accounts are optional, never required

A customer can place an order as a guest, exactly as before. Registering is offered so they can track delivery status and order history, but nothing behind checkout is gated on having an account.

Rule 3 — AI-assisted verification

Payment screenshots are checked automatically by an AI verification service before any balance is credited.

Rule 4 — Spend only what is verified

A customer can never spend more than the amount successfully verified and credited to their BoostX balance.

Rule 5 — Backend is the authority

Price, payment, balance, and order validation are decided by the backend. The frontend is for display only and is never trusted.

3. Customer Journey

End-to-end flow a first-time customer follows. An account is never required to order — registering only adds order tracking on top of the same flow:

1. Visit BoostX and select **New Order** — no sign-in prompt at this stage.
2. Select a platform (TikTok, Instagram, Facebook, X, or Telegram).
3. Select a service (e.g. TikTok Followers).
4. Enter the target link or username.
5. Enter quantity — BoostX shows a live price preview in GHS.
6. Pay the exact displayed amount via mobile money.
7. Upload a screenshot of the successful payment.
8. The AI verification service inspects the screenshot and confirms amount, recipient, status, and reference.
9. On success, the verified amount is credited to the customer's balance (guest session, or account if signed in).
10. The customer confirms the order; the backend re-checks balance and reserves the charge.
11. BoostX submits the order to the provider API and stores the returned provider order ID.
12. The customer tracks status (Pending → In Progress → Completed) using their BoostX order ID — or, if they register or sign in (Section 4), automatically inside their order history.

A guest who registers afterward doesn't lose their order history — see "Guest order claiming" in Section 4.

4. Accounts & Authentication

Customers can now register and log in to track deliveries and order history. Two rules shape everything in this section: accounts are optional (Rule 2), and customer accounts and admin accounts are never told apart anywhere the browser can see.

One shared login page

Customers and admins sign in through the exact same page, the exact same form (identifier + password), and the exact same API endpoint — `POST /api/auth/login`. There is no "Admin login" link, tab, toggle, or hidden query parameter anywhere on the public site. The only place a login is ever treated differently is **after** the password check succeeds, when the backend looks up that account's `role` column and decides where to send the browser next.

Outcome	What the browser is told
Customer credentials, correct	Redirected to <code>/account/orders</code> . No mention of role in the response.
Admin credentials, correct	Redirected to <code>/admin</code> . Same response shape as the customer case.
Wrong password	"Incorrect email/phone or password." — identical wording either way.
Account doesn't exist at all	The exact same "Incorrect email/phone or password." message — so the page can't be used to check which emails have accounts.
Account temporarily locked	Also the exact same generic message, so a locked-out attacker can't tell they found a real account and just need to wait.

Registration creates customers only

POST /api/auth/register hardcodes the new account's role to customer — there is no field, header, or parameter on that endpoint that can produce an admin account. Admin accounts are created exclusively by a separate function that is never wired to a public route: either a one-time seed script, or an internal endpoint that itself requires an already-authenticated admin session to call.

Login security (this matters more here than usual)

- Passwords are hashed (PBKDF2/bcrypt) — never stored or logged in plain text.
- 5 failed attempts locks the account for 15 minutes; the lockout is invisible to the person trying — they still just see the generic error.
- Session/role state is issued server-side as a signed token or server session; the browser can never edit its own role by tampering with a cookie or request body.
- Rate limiting applies to the login endpoint itself, independent of the per-account lockout, since this one endpoint is also the admin's front door.

Guest order claiming

Guest checkout is unchanged (Section 7 keeps the guest_sessions table). When someone who already ordered as a guest later registers — or logs in — with the same phone number or email they used at checkout, BoostX automatically re-attaches those past orders to the new account, so order history doesn't start from zero. This only matches the contact details the person just typed into the form in front of them; it never scans for or pulls in anyone else's orders.

```
register_customer(email, phone, password) -> account created
      |
      v
claim_guest_orders(user_id, email, phone)
      |
      v
any guest order whose session used this email/phone -> orders.user_id = account
```

Order tracking once signed in

Endpoint	Returns
GET /api/account/orders	Every order on the account: platform, service, target, quantity, status, delivered timestamp
GET /api/account/orders/summary	Counts: total placed, delivered, in progress, not delivered (canceled/failed/refunded)

Database additions

users

id, public_user_id, email, phone, password_hash, role, status, created_at, last_login_at, failed_login_attempts, locked_until

The existing orders table gains a nullable user_id column (foreign key to users). A guest order has user_id = NULL and only a session_id; a claimed or account-placed order has both.

Implementation reference: models.py, auth_service.py, routes.py, order_history_service.py, claim_service.py — see Appendix A.

How this differs from a typical SMM panel account model

A reference storefront checked directly (baloonboost.com/order) uses the account-first model common to most SMM panels: registration and sign-in are the normal way in, with order tracking and balance living inside a personal cabinet. BoostX inverts that by design:

	Typical panel (e.g. BalloonBoost)	BoostX
To place an order	Account required up front	Guest checkout — no account needed
To track orders	Personal cabinet, requires login	BoostX order ID works with no login; account optional for full history
Balance	Held in the account's cabinet	Held per guest session, or per account if signed in — same ledger either way

The practical result: **New Order** → **choose service** → **pay** → **upload proof** → **order** → **track** works start to finish without ever forcing registration. Accounts exist purely to add order history and delivery tracking on top of that same flow (Rule 2).

5. System Architecture

BoostX is a three-tier system: a frontend for guest ordering, a backend that owns all business rules, and a PostgreSQL database as the single source of truth for money and orders.

Customer

- > BoostX Frontend (Next.js / React)
- > BoostX Backend API (FastAPI or Flask)
 - > Pricing Engine (GHS pricing, flat markup)
 - > Payment Verification (AI vision + business rules)
 - > Ledger Service (append-only balance history)
 - > Order Service (validation, reservation)
 - > Provider Adapter (boostcenter2.com/api/v2)
 - > Admin Service
- > PostgreSQL (orders, payments, ledger, services)
- > Redis + background worker (status sync, refills, cleanup)
- > SMM Provider API (fulfillment)
- > AI Vision Service (payment screenshot analysis)

Component responsibilities

Component	Owns	Never does
Frontend	UI, forms, price preview, file upload, theme, tracking display	Never holds provider API keys, admin secrets, or the final price/balance authority
Backend API	All validation: service, target, quantity, price, balance, order	Never trusts a price, balance, or service ID sent from the browser
Pricing Engine	Converts provider USD cost to GHS and applies the flat markup	Never exposes the provider's wholesale rate to the customer
Provider Adapter	Talks to boostcenter2.com/api/v2 only	Never lets a provider error reach the customer unfiltered
Background Worker	Service sync, order-status polling, refills, session cleanup	Never makes the customer wait on a live provider round-trip

6. Technology Stack

Layer	Choice	Why
Frontend	Next.js / React (or React + Vite)	Fast guest-checkout UI, easy theming for light/dark mode
Backend	FastAPI (or Flask)	API-driven platform; FastAPI gives async I/O for provider polling
Database	PostgreSQL	ACID transactions — required for the ledger and atomic balance checks
Cache / Queue	Redis	Backs the background worker and rate limiting
Background jobs	Celery or RQ	Service sync, order-status polling, refills, cleanup
AI verification	Vision-capable AI API	Reads payment screenshots: amount, recipient, status, reference
Fulfillment	boostcenter2.com/api/v2	Confirmed live provider API — see Section 9
File storage	S3-compatible storage	Payment screenshots, stored outside public execution paths

7. Database Schema

PostgreSQL tables. Every balance movement is a ledger row — the system never stores a bare running total.

users

`id`, `public_user_id`, `email`, `phone`, `password_hash`, `role`, `status`, `created_at`, `last_login_at`, `failed_login_attempts`, `locked_until` — see Section 4

guest_sessions

`id`, `session_token`, `ip_hash`, `device_hash`, `phone`, `email`, `status`, `created_at`, `expires_at`, `last_seen_at`

platforms

`id`, `name`, `slug`, `icon`, `enabled`, `created_at`

services

`id`, `platform_id`, `provider_service_id`, `name`, `description`, `category`, `provider_rate`, `customer_rate`, `minimum`, `maximum`, `refill_available`, `enabled`, `sort_order`, `updated_at`

payments

`id`, `payment_reference`, `session_id`, `expected_amount`, `detected_amount`, `verified_amount`, `currency`, `payment_method`, `recipient_name`, `transaction_reference`, `screenshot_path`, `status`, `verification_score`, `created_at`, `verified_at`

ledger_transactions

`id`, `session_id`, `type`, `amount`, `balance_before`, `balance_after`, `reference_type`, `reference_id`, `status`, `created_at`

orders

id, public_order_id, session_id, user_id (nullable, added for accounts), service_id, provider_order_id, target, quantity, unit_price, charge, currency, status, start_count, remains, created_at, updated_at, completed_at

order_events

id, order_id, event_type, old_status, new_status, metadata, created_at

admin_actions

admin_id, action, object, old_value, new_value, timestamp, ip_address

Payment status values

pending · verified · rejected · review · expired

Ledger transaction types

payment_credit · order_debit · refund_credit · adjustment

User roles / statuses

roles: customer · admin — statuses: active · suspended

8. Pricing Engine — GHS Only

BoostX prices exclusively in Ghana Cedis. The provider's USD cost is converted to GHS and a flat markup of **GHS 5.00** is added once per order — not per unit, and not as a percentage. The formula:

```
customer_price_ghs =
  (provider_rate_usd_per_1000 / 1000 * quantity * usd_to_ghs_rate)
  + flat_markup_ghs    # default GHS 5.00
```

Worked example

Input	Value
Provider service	TikTok Followers (provider service ID 1)
Provider rate	\$0.90 per 1,000
Quantity ordered	1,000
USD → GHS rate (admin-configured)	10.70
Provider cost in GHS	GHS 9.63
Flat markup	+ GHS 5.00
Customer price shown	GHS 14.63

Rules enforced by the pricing engine

- Runs **server-side only**. Any price shown by the frontend is a preview; the backend recalculates before charging.
- The provider's wholesale USD rate is never returned to the customer — only the final GHS price.
- Quantity is validated against the service's provider-defined minimum/maximum before a quote is produced.
- Money values use fixed-point Decimal arithmetic, rounded half-up to 2 decimal places — never floating point.

USD → GHS exchange rate

The USD/GHS rate is **volatile** — quotes checked across multiple sources in the same week ranged from roughly GHS 10.70 to GHS 15.18 per USD. This rate must be an admin-configurable setting (or refreshed automatically from a live FX feed on a schedule), never hardcoded. A stale rate silently erodes margin or overcharges customers.

Implementation reference: `pricing_service.py` — see Appendix A.

9. Provider API Integration

BoostX's fulfillment provider is **boostcenter2.com/api/v2**. This was checked directly against the provider's published documentation. It is a standard SMM-panel API: a single POST endpoint, JSON responses, one action parameter per call.

Field	Value
HTTP Method	POST
API URL	https://boostcenter2.com/api/v2
Response format	JSON
Auth	key parameter (your provider API key) sent on every request

Actions

action	Purpose	Key parameters
services	Fetch the full service catalog (rate, min, max, refill flag)	key, action
add	Create an order	key, action, service, link, quantity, runs*, interval*
status	Check one order's status	key, action, order
status (bulk)	Check multiple orders at once	key, action, orders (comma-separated)
refill	Request a refill on an order	key, action, order
refill_status	Check a refill's status	key, action, refill
cancel	Cancel an order	key, action, order
balance	Check BoostX's balance held with the provider	key, action
get-username	Confirm the authenticated provider account	key, action

BoostX adapter mapping

Each provider action is wrapped by exactly one method on ProviderAdapter (Appendix A), so the rest of the backend never constructs a raw provider request:

```

get_services()           -> action=services
create_order(...)        -> action=add           -> provider_order_id
get_order_status(id)     -> action=status       -> charge, status, remains
get_multiple_order_status([]) -> action=status (bulk) -> {id: status}
request_refill(id)       -> action=refill       -> refill_id
get_refill_status(id)    -> action=refill_status -> status
cancel_order(id)         -> action=cancel       -> bool

```

```

get_provider_balance()    -> action=balance    -> (amount, currency)
get_username()            -> action=get-username -> username

```

Operational rules

- The provider API key lives only in a server-side environment variable — never in frontend code.
- Every provider call has a timeout and a small number of retries for network failures; provider-returned business errors (e.g. an invalid order ID) are not retried.
- The service catalog is **synced periodically by a background worker into PostgreSQL** — the customer-facing dropdown never calls the provider live, keeping the site fast and within the provider's rate limits.
- Order-status polling for pending orders should run on a schedule (roughly every 1–5 minutes, tuned to the provider's limits), not on every page load.

Reference checked separately: baloonboost.com/order is another provider-style storefront and was reviewed only for UI/platform-list reference — it is not BoostX's fulfillment provider.

Implementation reference: provider_client.py — see Appendix A.

10. Payment & AI Verification Flow

The customer pays by mobile money, uploads a screenshot, and the AI verification service extracts structured data before any balance is credited.

Payment account (configured, live)

Field	Value
Method	Telecel Cash
Account name	BOOSTX
Account number	0202979378

The payment page always shows the exact GHS amount to send and states it must be sent in full — any cross-network fee is added on top by the sender's own network, not deducted from what BoostX receives (see below). This value is admin-editable (Section 13); it is never hardcoded in the frontend.

Instructions shown to the customer, by their own network

The payment screen shows steps for the customer's own sending network only — either auto-detected from a phone number they've entered, or picked from three tabs (Telecel / MTN / AirtelTigo). Same-network Telecel payments have no fee; sending from MTN or AirtelTigo is a cross-network transfer and carries a small fee charged to the sender.

Sending from Telecel Cash — same network, no fee

1. Dial *110#
2. Select Send Money
3. Enter 0202979378
4. Enter the amount
5. Enter PIN to confirm

Sending from MTN Mobile Money — cross-network fee applies

1. Dial *170#
2. Select Transfer Money
3. Select Other Networks
4. Select Telecel

5. Enter 0202979378
6. Enter the amount
7. Confirm the recipient name shown (should read BOOSTX)
8. Enter PIN

Sending from AirtelTigo Money — cross-network fee applies

1. Dial *110#
2. Select Send Money
3. Select To Mobile Number
4. Select Other Networks → Telecel
5. Enter 0202979378
6. Enter the amount
7. Enter PIN

Cross-network transfers usually land within seconds to a few minutes. If a payment isn't credited within 10 minutes, the customer should check their SMS confirmation before retrying — cross-network MoMo transfers are final and cannot be auto-reversed, so the recipient number and name are worth confirming before sending.

The customer's SMS confirmation is exactly what the payment-screenshot upload should capture — amount, recipient name, and reference are the fields Section 10's AI verification checks below.

What the AI extracts

Amount · Recipient/account name · Transaction/reference · Payment status · Date/time · Payment provider

Checks the backend applies to the AI result

1. Amount — does the screenshot show a valid payment amount?
2. Recipient — does the account name match BoostX's configured payment account?
3. Transaction status — does the transaction show as successful?
4. Reference — is there a transaction/reference number?
5. Date/time — is the transaction recent enough to still be valid?
6. Duplicate detection — has this reference already been used?
7. Screenshot integrity — any inconsistencies or missing information?
8. Provider verification — where a real payment-gateway API is available, verify against it rather than the screenshot alone.

A screenshot is evidence, not cryptographic proof of payment. Unverifiable or suspicious transactions must be held as `review_required` rather than auto-credited.

The non-negotiable separation

AI result → backend business rules → ledger transaction. The AI service never writes directly to the balance.

Partial and over-payment policy

Scenario	Rule
Detected amount < expected	No silent full credit. Either reject, or credit exactly the detected amount if partial-credit is enabled.
Detected amount > expected	The extra amount is never silently spent — apply a defined policy (credit the full verified amount, or hold for review).
Duplicate transaction reference	Second use of the same reference is rejected: "Transaction already used." Enforce a DB unique constraint on the reference.

11. Order Lifecycle & Money Safety

Final order validation (on "Confirm Order")

1. Retrieve the service and confirm it is active
2. Verify the platform is enabled
3. Validate the target format
4. Validate quantity against min/max
5. Retrieve the current selling price from the pricing engine
6. Retrieve the customer's verified ledger balance
7. Check available balance $\geq$ charge
8. Reserve/deduct the amount atomically
9. Submit the order to the provider
10. Save the returned provider order ID
11. Create the BoostX order record
12. Return the order confirmation

If any step fails, reserved money is restored to the customer's balance. Money is never lost between reservation and provider confirmation.

Anti-race-condition rule

Example: balance GHS 50. Two browser tabs each submit a GHS 40 order at the same moment. The database transaction must lock/check the balance atomically so only one order can succeed — the balance must never go negative.

Idempotency

If "Confirm Order" is clicked three times on a slow connection, BoostX must create exactly one provider order. Use an idempotency key (e.g. BX-IDEMP-839201); a repeated request returns the original order instead of creating a new one.

Provider failure handling

Event	Result
Provider call fails after reservation	Reserved amount is released back to the customer's balance; event PROVIDER_ORDER_FAILED is recorded.
Provider call succeeds	Ledger moves from reserved to debited; order status becomes active.

Order statuses

Pending · In Progress · Completed · Partial · Canceled · Failed · Refunded

12. Reliability Principle — What BoostX Defends Against

No real system can honestly guarantee zero issues — a provider will time out, a screenshot will be blurry, a customer will double-click. BoostX's design goal is narrower and achievable: **common failures should never cost the customer their money or leave an order in an undefined state**. The table below is a single index of where that promise is actually enforced elsewhere in this document — nothing here is new behavior, it is a deliberate summary of it.

Failure scenario	What BoostX does about it	Where
Customer double-clicks "Confirm Order"	An idempotency key ensures exactly one provider order is created; repeats return the original order.	Section 11
Provider API fails after money was reserved	The reserved amount is released back to the customer's balance; the failure is logged as an event.	Section 11
Two orders submitted at the same instant	The balance check and deduction are atomic — only one order can succeed; the balance never goes negative.	Section 11
Fake, edited, or duplicate payment screenshot	AI verification checks amount, recipient, reference, status, and date, and rejects a reused reference outright.	Section 10
Provider API key leaking to the browser	The key is a server-side-only environment variable; the frontend never sees or sends it.	Sections 9, 15
A provider service behaves differently than documented	Every provider response is validated and wrapped before use; an unexpected shape raises a handled error, never a silent bad charge.	Section 9

What this section does not claim

This is deliberately not a claim that BoostX will run issue-free. It is a claim that the specific failure modes above — the ones that would otherwise cost a customer money or leave an order unaccounted for — are each handled by a concrete, testable rule rather than left to chance. New failure modes will surface in production that aren't on this list yet; Section 17's pre-launch validation step exists precisely to find them before customers do.

13. Admin Dashboard

Administrators sign in through the exact same shared login page as customers (Section 4) — there is no separate /admin/login page to discover. A correct admin login simply redirects to /admin instead of the customer order-tracking view; everything past that point (2FA, rate-limited login, admin session) applies as before.

Area	Admin can
Overview	See today's orders, today's revenue, pending/verified/failed payments, active orders, provider balance
Payments	Review payment ID, session, expected vs AI-detected amount, recipient, reference, AI result, and manually resolve review-required cases
Services	Enable/disable a service; change customer price, minimum, maximum, description, category, refill availability

Area	Admin can
Platforms	Enable/disable TikTok, Instagram, Facebook, X, Telegram — no others are shown to customers
Orders	View, track, cancel (where supported), refill (where supported), view events, view the linked payment
Payment settings	Configure payment method, account name/number, instructions, currency, min/max payment, AI verification rules — configured today as Telecel Cash, BOOSTX, 0202979378 (Section 10)
Provider settings	Configure provider name, API URL, API key (server-side secret only), markup, sync interval
Audit log	Every admin action recorded: admin ID, action, object, old value, new value, timestamp

14. API Endpoints

Authentication (shared by customers and admins)

```

POST /api/auth/register      # always creates a customer account
POST /api/auth/login         # single endpoint for BOTH roles
POST /api/auth/logout
GET  /api/auth/me            # role known only after a session exists

```

Customer-facing

```

GET /api/platforms
GET /api/services
GET /api/services/:id
POST /api/session
POST /api/payments
POST /api/payments/:id/screenshot
GET /api/payments/:id
GET /api/balance
POST /api/orders/preview
POST /api/orders
GET /api/orders/:public_id
GET /api/orders/:public_id/status
POST /api/orders/:public_id/refill
POST /api/orders/:public_id/cancel

```

Account-holder only (requires a signed-in session)

```

GET /api/account/orders
GET /api/account/orders/summary

```

Admin

```

GET /api/admin/dashboard
GET /api/admin/payments
GET /api/admin/orders
GET /api/admin/services
PATCH /api/admin/services/:id
GET /api/admin/settings
PATCH /api/admin/settings
GET /api/admin/audit-log
POST /api/admin/admins      # create an admin; caller must
                             # already be an authenticated admin

```

Note: there is no POST /api/admin/login — admins authenticate through POST /api/auth/login, the same endpoint customers use (Section 4).

15. Security Requirements

Never trusted from the browser

price · quantity · service ID · balance · role — the backend always re-derives these itself.

Secrets — server-side only. never in frontend code

```
DATABASE_URL
PROVIDER_API_URL
PROVIDER_API_KEY
AI_API_KEY
SECRET_KEY
ADMIN_SECRET
PAYMENT_ACCOUNT_NAME
PAYMENT_ACCOUNT_NUMBER
PAYMENT_METHOD
CURRENCY=GHS
```

File upload security

- Accept image formats only (PNG, JPG, JPEG, WEBP); enforce a maximum file size.
- Never use the customer's original filename as a storage path.
- Rename internally, e.g. /payments/2026/09/BX-PAY-839201-randomid.webp.
- Store outside publicly executable paths, with controlled access.

Platform-wide controls

- HTTPS everywhere; secure, expiring sessions
- CSRF protection where applicable
- Rate limiting on payment upload, order creation, order tracking, the shared login endpoint, and AI verification
- Input validation and file-type/size validation on every upload
- SQL injection protection via parameterized queries / ORM
- Audit logs for every admin action; request logging with no secrets in logs
- Idempotency keys on order creation; unique constraint on transaction references

Shared login page hardening

- Identical error message for wrong password, unknown account, and locked account — no enumeration.
- Account lockout after 5 failed attempts (15 minutes) — enforced per account, not just per IP.
- Passwords hashed with PBKDF2/bcrypt; never logged, never returned in any API response.
- Role is read from the database after authentication, never accepted as input from the client.
- Registration can only ever produce a customer account; admin accounts require an already-authenticated admin.

16. MVP Build Phases

Phase 1 — Launch scope

- Branding, homepage, New Order flow, five platforms, service selection, GHS pricing
- Guest sessions, payment instructions, screenshot upload, AI verification, ledger
- Order creation, provider API integration, order tracking
- Light/dark mode

- Customer registration and login, shared with admin login, order history and delivery tracking, guest order claiming
- Admin dashboard, PostgreSQL

Phase 2 — Near-term additions

- Refills, cancellation, refund automation
- Email and SMS/WhatsApp notifications
- Advanced analytics
- Multiple payment methods

Future — Not in MVP

- Customer accounts and wallets, saved targets, order templates
- Promo codes, referral system, loyalty points
- Multiple currencies, multiple providers with automatic failover
- Advanced fraud detection, mobile app

MVP is successful when a brand-new customer can

Visit BoostX → pick a platform and service → enter target and quantity → see the GHS price → pay → upload a screenshot → get AI-verified balance → place the order with no account → have it submitted through the provider API → track it to completion → receive a refund/credit if it fails — all on mobile or desktop, light or dark mode.

17. Critical Test Cases

These four scenarios must pass before launch — they protect real money:

#	Scenario	Expected result
1	Balance GHS 50. Two simultaneous orders of GHS 40 each.	Exactly one order succeeds; the other is rejected. Balance never goes negative.
2	Expected payment GHS 50; screenshot shows GHS 20.	No GHS 50 credit is given. Only the verified/policy-defined amount is credited.
3	Same transaction reference (e.g. ABC123) uploaded twice.	Second attempt is rejected: "Transaction already used."
4	Balance GHS 50; order GHS 30 is placed; provider call fails.	Customer's balance returns to GHS 50 — not left at GHS 20.

Also verify

- Frontend and backend on mobile, tablet, desktop, light mode, and dark mode
- Form validation, upload validation, order tracking by public order ID
- Refund logic on cancel/fail/partial
- Admin authorization boundaries
- API-key exposure checks — confirm no secret appears in any frontend bundle or client response
- Shared login: wrong password, unknown account, and a locked account all return the identical error
- The register endpoint cannot be made to produce an admin account under any input
- Guest order claiming links only orders matching the contact details entered at registration/login — never someone else's

Pre-launch validation — provider service mapping

Section 9 confirms the shape of `boostcenter2.com/api/v2` — the actions, parameters, and response fields. That is not the same as confirming every individual service behaves correctly. Before launch, each of the five platforms' live services needs its own pass:

- Call services for real and confirm every service BoostX intends to sell is actually present, priced, and within a sane min/max.
- Place one small real order per platform end-to-end (pay → verify → submit → track) and confirm the provider order ID, status transitions, and final delivery all match what the customer sees.
- Confirm refill and cancel behave as documented for the specific services that advertise them — not every provider service supports both, regardless of what the catalog flag says.
- Re-run the four money-safety test cases above against the real provider, not a mock — provider failure handling (Section 11) is only proven once it's been triggered against the live API.

This step is what turns “the API was checked” into “BoostX is safe to launch” — the two are not the same claim, and the gap between them is exactly what this checklist closes.

18. Environment Variables Checklist

```
DATABASE_URL=
PROVIDER_API_URL=https://boostcenter2.com/api/v2
PROVIDER_API_KEY=
AI_API_KEY=
SECRET_KEY=
ADMIN_SECRET=
PAYMENT_ACCOUNT_NAME=BOOSTX
PAYMENT_ACCOUNT_NUMBER=0202979378
PAYMENT_METHOD=Telecel Cash
CURRENCY=GHS
USD_TO_GHS_RATE=          # admin-editable; do not hardcode long-term
FLAT_MARKUP_GHS=5.00
```

Even though the payment account is now known, it stays an environment variable rather than a hardcoded string — Section 13's admin settings screen reads and writes this same value, so it can change without a redeploy.

Appendix A. Code Delivered With This Specification

Seven backend modules implement Sections 4, 8, and 9 of this document and were provided alongside this PDF, across three prior turns:

File	Implements
<code>pricing_service.py</code>	PricingService / PricingSettings — GHS conversion + flat GHS 5.00 markup, Decimal-safe rounding, quantity-range validation (Section 8)
<code>provider_client.py</code>	ProviderAdapter — full <code>boostcenter2.com/api/v2</code> coverage: services, add, status (single + bulk), refill, refill_status, cancel, balance, get-username, with retries and error wrapping (Section 9)
<code>models.py</code>	User model, roles (customer/admin), password hashing, lockout fields (Section 4)
<code>auth_service.py</code>	AuthService — unified <code>authenticate()</code> , customer-only <code>register_customer()</code> , separate <code>create_admin_account()</code> , generic error messages, lockout logic (Section 4)
<code>routes.py</code>	Flask blueprint: shared POST <code>/api/auth/login</code> , POST <code>/api/auth/register</code> , logout, <code>/me</code> (Section 4, Section 14)

File	Implements
order_history_service.py	OrderHistoryService — per-account order list and delivered/in-progress/not-delivered summary (Section 4)
claim_service.py	ClaimService — re-attaches a customer's past guest orders on registration/login by matching email or phone (Section 4)

End of document — BoostX MVP Build Specification.